2. (a) Convert the following numbers as indicated:

(i) $(41.6875)_{10}$ → Binary.

2	41	
2	20	1
2	10	0
2	5	0
2	2	1
	1	0

$0.6875 \times 2 = 1.375 \Rightarrow 1$

$0.375 \times 2 = 0.75 = 0$

$0.75 \times 2 = 1.5 = 1$

$0.5 \times 2 = 1.0 = 1$

$= (101001.1011)_2$ Ans

(ii) $(101101.101)_2$ → Decimal

$10^{5}\,1^{4}\,1^{3}\,0^{2}\,1^{1}\,1^{0} \cdot 1^{-1}\,0^{-2}\,1^{-3}$

$(1\times2^5 + 0\times2^4 + 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0) \cdot (1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3})$

$\Rightarrow 32 + 0 + 8 + 4 + 0 + 1 \cdot 5 + 25 + 125$

$= (45.875)_{10}$ Ans

(iii) $(736.4)_8$ → Decimal

$(7^{2}\,3^{1}\,6^{0}.4^{-1})_8 \Rightarrow 7\times8^2 + 3\times8^1 + 6\times8^0 \cdot 4\times8^{-1}$

$\Rightarrow 448 + 24 + 6 \cdot 5$

$(736.4)_8 = (478.5)_{10}$ Ans

(iv) $(7A3.F)_{16}$ → Binary

$(7A3.F)_{16} = (0111\ 1010\ 0011 \cdot 1111)_2$

Scanned with OKEN Scanner

2. (b) Perform the following conversions:

(i) Convert the decimal number 458 into octal-coded decimal (OCD).

Sol:- Octal range: 0-7.

Decimal digit	Octal	Binary (3bit)
4	4	100
5	5	101
8	10 (octal)	001 000

OCD : 458 is 100 101 001 000

(ii) Convert the decimal number 789 into Hexa-coded decimal (HCD)

Sol:- 789 in HCD

7	8	9
0111	1000	1001

(iii) Convert the decimal number 259 into Binary coded decimal (BCD)

Sol:- 259 in BCD

2	5	9
0010	0101	1001

Scanned with OKEN Scanner

3. (a) Using 2's complement method, perform the following subtractions:

(i) 1010100 − 1000011

So: Subtrahend : 1000011

Minuend : 1010100

Step1: Perform 2's complement to the subtrahend.

```
  1000011
  0111100   ← 1's complement
        1   ← +1 (LSB) → 2's comp.
-------------
  0111101   (2's complement)
```

Step2: Now, add the 2's complement number and the Minuend.

```
    ①①①
  1010100   (Minuend)
+ 0111101   (2's complement)
-------------
 [1]0010001
```

Step3: Carry is ignored; for 2's complement Subtraction if the result is a positive number..

So, the result is 0010001 which is equal to the decimal no.(17.)

Verification: $(1010100)_2 = (84)_{10}$

$(1000011)_2 = (67)_{10}$

$84 - 67 = (17)_{10}$

Scanned with OKEN Scanner

(ii) 100101 − 110011

Sol:

Subtrahend : 110011

Minuend : 100101

Step:- Perform 2's complement to the Subtrahend.

```
  110011
  001100  <- 1's complement
+      1  <- +1 (LSB)
  -------
  001101  <- (2's complement)
```

Step2:- Now, add this 2's complement to the minuend.

```
minuend  (1) (1)  (1)
         100101   (minuend)
+        001101   (2's complement)
  -------------
  | 110010 |  <- 2's complement form (-14)
  -------------
```

For checking: 100101 => $(37)_{10}$

110011 => $(51)_{10}$

37 − 51 = −14.

−14 in 2's complement form.

14 : 001110

```
  | 110010 |  2's complement
  ----------
```

Scanned with OKEN Scanner

3. (b) Represent the following in fixed-point magnitude and 2's complement (8-bit).

(i) +25

Sol:- +25

25 in binary is $(11001)_2$

2 | 25
2 | 12 - 1
2 | 6 - 0
2 | 3 - 0
1 - 1

8-bit format:-

Sign is '+' => MSB = 0.

0 0011001 → Fixed point magnitude (8bit)

↑ sign

0 0011001 → 2's complement (It's a positive value no need to perform complement)

(ii) -25

Sol:- Value (25)$_{10}$ = $(11001)_2$

Step1: Now, the sign is negative. MSB = 1 in an 8-bit format

1 0011001 → Fixed point magnitude (8bit)

↑ negative

Step2: Perform 2's complement.

25 :→ 0 0 0 1 1 0 0 1

1 1 1 0 0 1 1 0 i's comp

1 +1

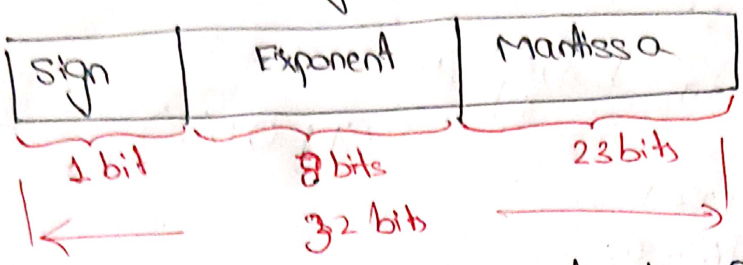
1 1 1 0 0 1 1 1 (2's comp)

Scanned with OKEN Scanner

4. (b) Represent the following numbers in IEEE-754 single precision format : (6)

(i) 12.5

IEEE 754 Single precision.



Step1:- Check the given number is positive or negative. In our example, it's a positive number. So, assign a '0' bit in the Sign place.

∴ Sign = 0

Step2:- Now, Convert 12.5 to binary.

12 => $(1100)_2$

0.5 => 0.5 x 2 = 1.0 => 1

∴ $12.5 = (1100.1)_2$

Step 3:- It's time to normalize the number. For this, need to move the binary point after the first '1'.

i.e, $1100.1 \Rightarrow 1.1001 \times 2^3$

(moved '3' positions)

∴ Exponent = 3

Step4:- Computer will recognize mostly positive exponentially. So, add bias to get the stored exponent.

Stored exponent = Actual exponent + Bias.

Bias = $2^{K-1} - 1$

where; K => Range of the exponent

= 8

∴ Bias = $2^{8-1} - 1 \Rightarrow 2^7 - 1 = 127$

Actual exponent = 3.

∴ Stored exponent = 3 + 127

= 130

{P.T.O}

Scanned with OKEN Scanner

Convert this $(130)_{10}$ into binary

$(10000010)_2$

This is the exponent value.

2 | 130
2 | 65 - 0
2 | 32 - 1
2 | 16 - 0
2 | 8 - 0
2 | 4 - 0
2 | 2 - 0
1 - 0

Step 5:. For finding the mantissa take the bits after the leading 1.

1.1001

Don't Consider this '1'

↳ Mantissa

∴ Mantissa = 1001

Size of mantissa according to the IEEE 754 Single precision is 23 bits (Fill remaining bit to '0')

1.00100000000000000000

23 bits.

Step 6:. IEEE 754 Single precision format.

Sign	Exponent	Mantissa.
0	10000010	00100000000000000000000

Scanned with OKEN Scanner

(ii) 263.3

Sol: Follow same steps as previous one:

2 | 263	
2 | 131 - 1	
2 | 65 - 1	
2 | 32 - 1	
2 | 16 - 0	
2 | 8 - 0	
2 | 4 - 0	
2 | 2 - 0	
1 - 0	

$263 = (100000111)_2$

$0.3 \times 2 = 0.6 = 0$

$0.6 \times 2 = 1.2 = 1$

$0.2 \times 2 = 0.4 = 0$

$0.4 \times 2 = 0.8 = 0$

$0.8 \times 2 = 1.6 = 1$

$0.6 \times 2 = 1.2 = 1$

$\therefore 263 = 100000111.010011\ldots$

Normalize

$1.00000111010011\ldots \times 2^8$

Exponent = 8

Stored exponent = $8 + 127 = 135$

2 | 135	
2 | 67 - 1	
2 | 33 - 1	
2 | 16 - 1	
2 | 8 - 0	
2 | 4 - 0	
2 | 2 - 0	
1 - 0	

$135 = 10000111$

Mantissa

00000111010011000000000

23 bits

Sign	Exponent	Mantissa
0	10000111	00000111010011000000000

8. (b) Multiply the following using Booth's algorithm
(+6) x (-5)

Sol: Step1: Multiplicand (M) = +6

+6 = 0110

Multiplier (Q) = -5

first represent 5 in binary then convert into 2's complement

5 = 0101

-5 = 1011 (2's complement)

Step2: Conditions for Booth's algo

$Q_0 Q_{-1}$	Operation
0 0	Arithmetic right shift
0 1	A ← A+m ; Arithmetic right shift
1 0	A ← A-m ; " "
1 1	Arithmetic right shift

Step 3: Initialization.

A → 0000

Q = 1011

Q_{-1} = 0

M = 0110

	A	Q	Q_{-1}		
Initial	0000	1011	0		A: 0000
					+m: 0110
					1010 (-6)
1st cycle	0000	1011	0	A ← A-m	A: 0000
	1010 → 1101	1011 → 0101	0 → 1	Arithmetic Right shift	-6: 1010
					A ← 1010
Find cycle	1101 → 1110	0101 → 1010	1 → 1	Arithmetic Right shift	

(P.T.O)

	A	Q	Q_{-1}	
Initial				(10)
3rd cycle	1110	1010	1	A: 1110 m: 0110
	0100	1010	1	A ← A+m
	0010	0101	0	A ← 0100
				Arithmetic right shift
4th cycle	0010	0101	0	A: 0010
	1100	0101	0	A ← A-m -m: 1010
	1110	0010	1	A ← 1100
				Arithmetic right shift

Finally: 1110 0010

Verification: $6 \times -5 = -30$.

16 8 4 2 1

$30 = 0001\ 1\ 1\ 1\ 0$

$-30 = 1110\ 0\ 0\ 1\ 0$

9. (b) Multiply the following Booth algorithm: $(-8 \times +7)$

So:

Multiplicand (M): -4

$4 = 0100$

$-4 = 1100$

Multiplier (Q): $+7$.

$+7 = 0111$

	A	Q	Q_{-1}	
Initial	0000	0111	0	
1st cycle	0000	0111	0	A: 0000
	0000	0111	0	-m: 1100
	0100	0111	0	A ← A-m
	0010	0011	1	A-(-4) : A+4
				A: 0000
				4: 0100
				A ← 0100
				Arithmetic right shift

(P.T.O)

Scanned with OKEN Scanner

	A	Q	Q-1		
1st cycle	0010	0011	1		
	0001	0001	1	Arithmetic right shift	
2nd cycle	0001	0000	1		
	0000	1000	1	Arithmetic right shift	
3rd cycle	0000	1000	1		
	1100	1000	1	A ← A+m	A: 0000, m (-4): 1100
	1110	0100	0	Arithmetic right shift	A ← 1100

Finaly: 1110 0100

Verification; +4 × −7 = −28

16 8 4 2 1

28 : 0 0 0 1 1 1 0 0

−28 : 1 1 1 0 0 1 0 0 (2's complement)

10. (b) Perform restoring division for 101101 ÷ 101

Sol:

Dividend (Q) = 10110 1

Divisor (M) = 101

A = Q+1 = 0000000 (7 bits)

	A	Q		
Initial	0000000	101101		m: 101
1st cycle	0000000	101101		d,
	0000000	011010	Left shift	0000010 1
	0000001	011010	A ← A-M	2's com. 11111011
	1111100	011010	check A<0	A: 0000001
				+1111011
				A ← 1111100
	0000001	011010	A ← A+m	m: 0000101
			Restore Q0 = 0	A ← 0000001

Scanned with OKEN Scanner

1st cycle

A	Q		
0000001	011010		
0000010	110100	Left shift	
(1)111101	110100	A←A+m	A: 0000010
		check A<0	-m: 1111011
0000010	110100	A←A+m	A←1111101
		Restore Q_0=0	0000101
			A←0000010

2nd cycle

0000010	110100		
0000101	101000	Left shift	A: 0000101
(0)0000000	101000	A←A-m	-m: 1111011
			0000000
0000000	101001	A≥0	
		Restore Q_0=1	

3rd cycle

0000000	101001		
0000001	010010	Left shift	A: 0000001
(1)111100	010010	A←A-m	-m: 1111011
			1111100
0000001	010010	A<0	m: 0000101
		A←A+m	A←0000001
		Restore Q_0=0	

4th cycle

0000001	010010		
0000010	100100	Left shift	A: 0000010
(1)111101	100100	A←A-m	-m: 1111011
			1111101
0000010	100100	A<0	m: 0000101
		A←A+m	A←0000010
		Restore Q_0=0	

5th cycle

0000010	100100		
0000101	001000	Left shift	A: 0000101
(0)0000000	001000	A←A-m	-m: 1111011
		A≥0	A←0000000
0000000	001001		
		Restore Q_0=1	

Result:

Final Result: A : 000 0000 [Remainder]
Q : 001001 [Quotient]

Verification: $101101 = (45)_{10}$
$101 = (5)_{10}$
$45 \div 5 = 9 \Rightarrow (1001)_2$

Scanned with OKEN Scanner